EXPERIMENT NO. 7

Title
Build an Interactive Form and Validate Inputs using JavaScript and DOM Manipulation

Aim
To design an interactive HTML form and perform client-side validation using JavaScript and DOM manipulation.

Software Requirements
Operating System: Windows / Linux / macOS
Tools: Web Browser (Chrome / Firefox)
Editor: VS Code / Notepad++

Introduction
JavaScript enables dynamic interaction with web pages. Using DOM manipulation, form elements can be accessed and validated before submitting data to the server.

Theory
Form validation ensures that user input meets required criteria. JavaScript DOM methods such as getElementById() allow accessing and modifying HTML elements dynamically.

Algorithm
1. Create an HTML form with input fields
2. Write JavaScript validation logic
3. Access form elements using DOM
4. Validate inputs on submit
5. Display error messages
6. Submit form if validation succeeds

Program Code
```
<!DOCTYPE html>
<html>
<head>
<title>Form Validation</title>
<script>
function validateForm() {
  var name = document.getElementById("name").value;
  var email = document.getElementById("email").value;
  if(name === "" || email === "") {
    alert("All fields are required");
    return false;
```

```
 }
 alert("Form submitted successfully");
 return true;
}
</script>
</head>
<body>
<form onsubmit="return validateForm()">
Name: <input type="text" id="name"><br><br>
Email: <input type="email" id="email"><br><br>
<input type="submit" value="Submit">
</form>
</body>
</html>
```

Output

Alert message displayed if fields are empty.
Success message displayed on valid input.
(Attach screenshot of browser output)

Learning Outcomes

Students will be able to design interactive forms and perform client-side validation using JavaScript and DOM.

Conclusion

Thus, an interactive form was successfully created and validated using JavaScript and DOM manipulation.